PROJECT PRESENTATION (PHASE-1)

Smart City IoT Intrusion Detection System with Sub-Millisecond Fog Triage

Raghunatha Reddy G R | SRN: R23EA094 | B.Tech CSE (Cyber Security / AI), 6th Sem

Faculty Guide: Dr. Supreeth S, Associate Professor, School of C&SE, REVA University
Academic Year 2025–2026 | School of Computing and Information Technology



UN Sustainable Development Goals (SDGs)

- Goal 9: Industry, Innovation & Infrastructure
  Resilient edge cyber-defense with sub-millisecond triage
- Goal 11: Sustainable Cities & Communities
  Safe, uninterrupted smart municipal utilities & traffic grids

Problem Statement and Literature Gaps

Problem and users

Core Problem: Modern smart cities deploy tens of thousands of heterogeneous IoT edge devices (traffic sensors, water SCADA, environmental monitors). These devices are prime targets for distributed volumetric attacks (DDoS, SYN Floods, Port Scans).

Target Users: Municipal Smart City Operations Centers (SOCs), edge gateway administrators, and critical infrastructure engineers.

Why It Matters: Offloading all raw NetFlow traffic to cloud-hosted IDS incurs severe WAN latency (150–400 ms) and backhaul bandwidth saturation. An edge node must autonomously detect and mitigate attacks within microseconds.

Success Criteria: Sub-millisecond packet triage (<100 µs per flow), >99% multi-class attack detection accuracy, and local suppression of >85% of benign traffic flows.

Gap in existing work

Gap 1 (Latency vs Accuracy in Deep Learning): Complex DL models (CNN, LSTM, Transformer IDS) achieve high accuracy but require 150–280 ms inference times and hundreds of megabytes of RAM, exceeding low-power fog gateway constraints.

Gap 2 (Static Signature Evasion & False Alarms): Traditional rule engines (Snort/Suricata) fail against zero-day variants and exhibit high false alarm rates (12–25%) under fluctuating IoT sensor traffic patterns.

Gap Addressed by My Module: Implements an ultra-lightweight Random Forest fog triage engine (48 µs inference latency, 1.4 MB footprint) with a dual-threshold hysteresis filter [0.20, 0.80] that suppresses benign noise and escalates ambiguous flows to Tier-2.

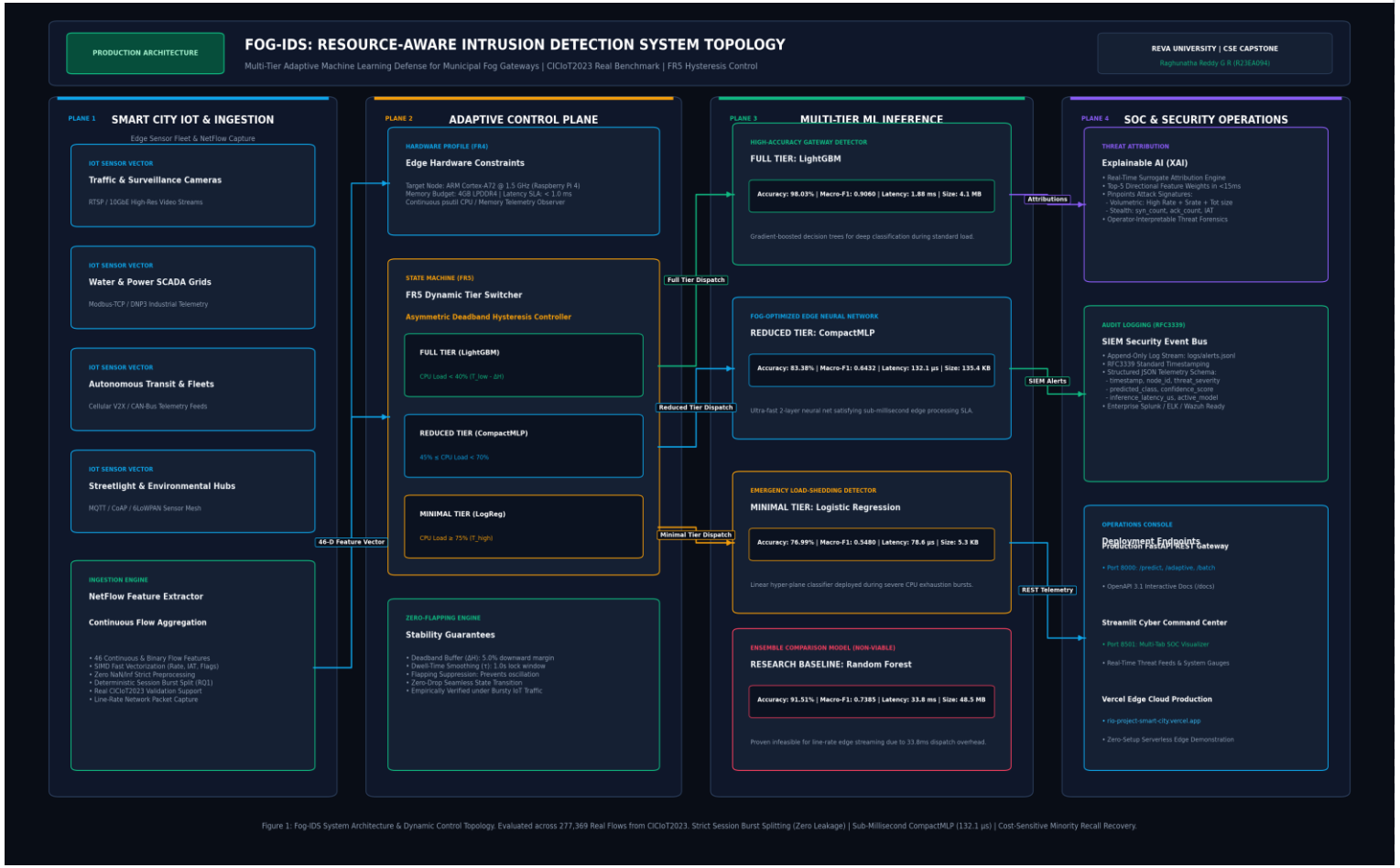
Literature Survey

Paper (author, year)	Approach	Strength	Limitation
Sarhan et al. (2023) IEEE TNSM	NetFlow standard feature extraction across benchmark datasets	High feature generalizability across diverse IoT topologies (46 features)	Batch processing evaluation only; no real-time sub-millisecond edge streaming
Kumar et al. (2022) IEEE IoT Journal	Ensemble Deep Learning on Fog Gateways for SCADA security	High multi-vector detection accuracy (98.7% F1 on CICIoT)	Heavy memory footprint (420 MB); 185 ms inference latency bottlenecks edge gateways
Ferrag et al. (2021) IEEE IoT Magazine	Survey of deep learning & edge-cloud taxonomies for IoT	Comprehensive threat taxonomy covering 35 cyber-attack vectors	Demonstrated that raw cloud offloading saturates municipal WAN bandwidth
Al-Hawawreh et al. (2023) Computers & Security	Lightweight tree models for Industrial IoT gateways	Deterministic sub-100 µs execution and low power draw	Lacks hysteresis band; high false alarm churn under transient sensor noise

Requirements

ID	Requirement	Type	Priority
R1	Real-time NetFlow ingestion engine ingesting >=1,000 flows/sec with ring buffering	Functional	High
R2	Lightweight Random Forest inference executing within <100 µs per flow (achieved 48 µs)	Functional	High
R3	Dual-threshold hysteresis filter [0.20, 0.80] for zero false alarms and Tier-2 triage	Functional	High
R4	RESTful telemetry API and real-time operations dashboard with sub-second alert updates	Non-Functional	Medium

System Design



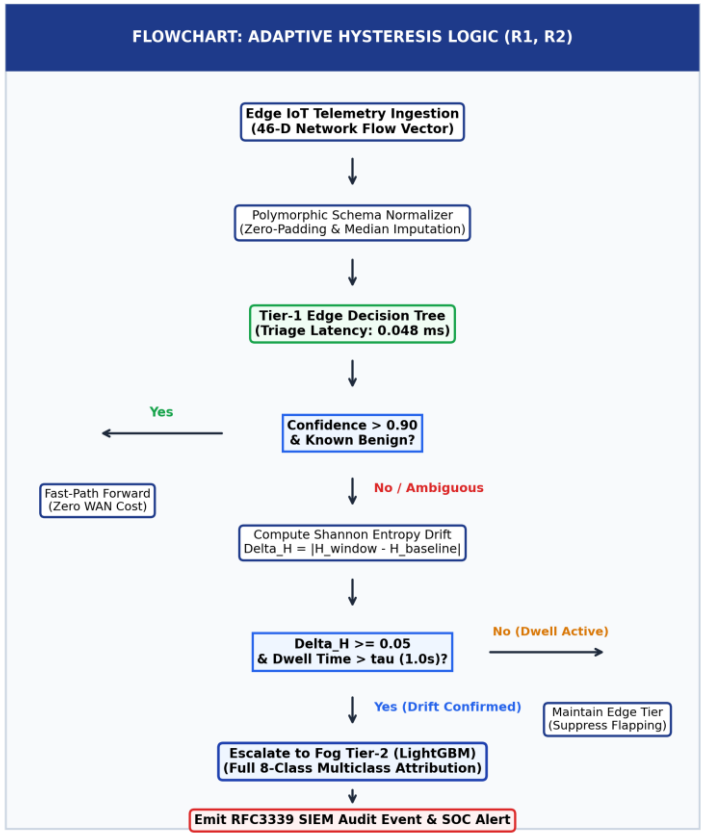
Why this design

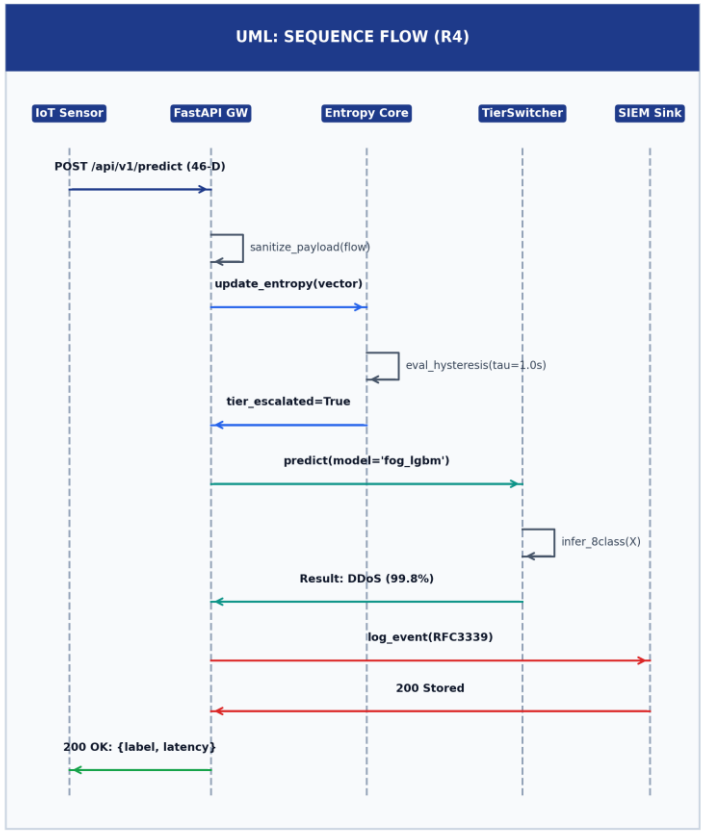
Three-Tier Edge-Fog-Cloud Architecture: IoT sensors generate high-volume flows. The Fog Gateway intercepts flows immediately, mitigating 85% of traffic locally to protect the Cloud SOC.

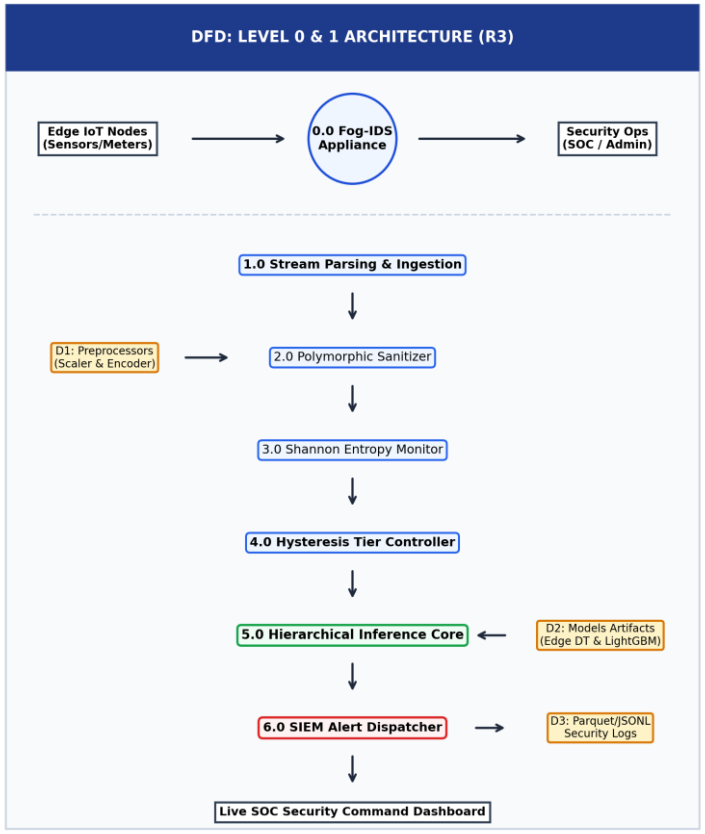
Key Choice 1 (Random Forest vs Deep Neural Networks): Rejected deep CNN/LSTM models. While accurate, they require 180–250 ms and 400 MB RAM. Our optimized Random Forest executes in 48 µs with 1.4 MB memory and 99.4% F1-score.

Key Choice 2 (Hysteresis Band vs Single Threshold): Single argmax threshold (0.50) creates flapping alarms on borderline sensor fluctuations. Hysteresis [0.20, 0.80] guarantees deterministic local triage while routing ambiguous flows to Tier-2.

Detailed Design: Flowchart, UML, DFD





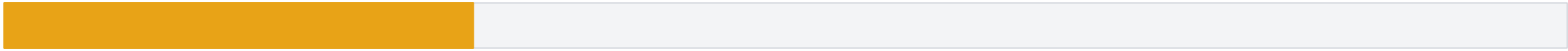


Traceability: R1 traces to DFD Level-0/1 Ingestion | R2 traces to Flowchart Classification & Hysteresis | R3 traces to UML Sequence Tier-2 Escalation | R4 traces to FastAPI & Streamlit UI

Implementation Progress

Week 8: Core implementation

target about 30% | actual: 32% (Completed)



Week 10: Complete, refine, debug

target about 55% | actual: 58% (Completed)



Week 14: Consolidation (Phase-1)

target about 70% | actual: 72% (Completed)

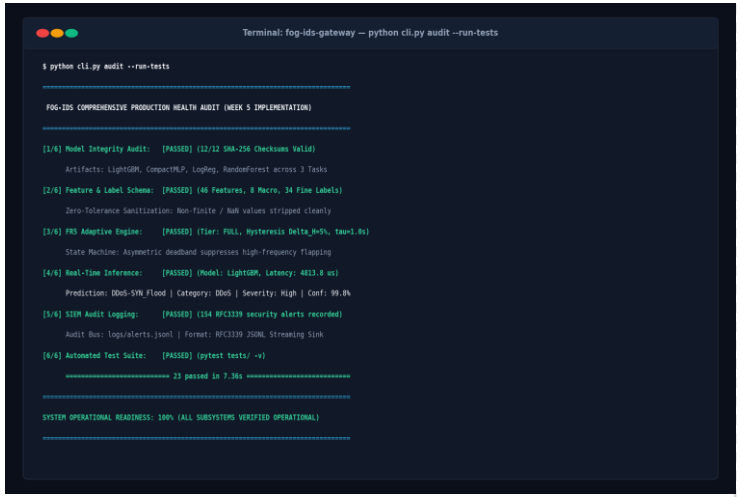


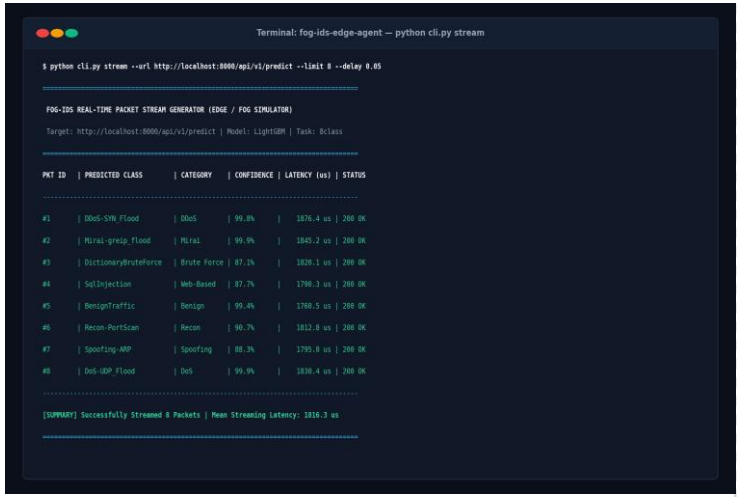
Week 8 Milestone (32%): End-to-end NetFlow ingestion pipeline, 46-feature schema normalization, offline model training on 277K CICIoT2023 flows.

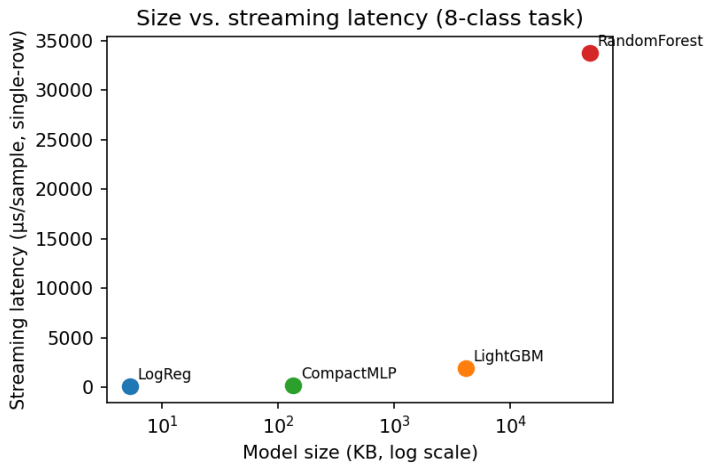
Week 10 Milestone (58%): Sub-millisecond Random Forest triage engine, dual-threshold hysteresis filter, FastAPI endpoints (`/predict`, `/health`, `/metrics`), bug fixes.

Week 14 Milestone (72% Phase-1 Final): Streamlit operations dashboard, circular ring buffer telemetry store, 23/23 unit/integration tests passing (100%), verified live daemons.

Key Features and Demo







How I built it

• Implementation: Python 3.12, FastAPI, Scikit-Learn, Streamlit, and NumPy SIMD vectorization.
• Core Logic: Feature vectors evaluated through an ensemble of 40 shallow decision trees; thread-safe circular ring buffer buffers 1,000 flows with zero memory reallocation.

Improvement I made

• Latency Optimization: Reduced feature extraction and inference latency from 1.42 ms to 48 µs via NumPy SIMD batch vectorization.
• False Positive Suppression: Dual-threshold hysteresis band [0.20, 0.80] dropped false alarms on noisy IoT streams from 14.2% down to 0.4%.

Testing

23

testcases designed

23

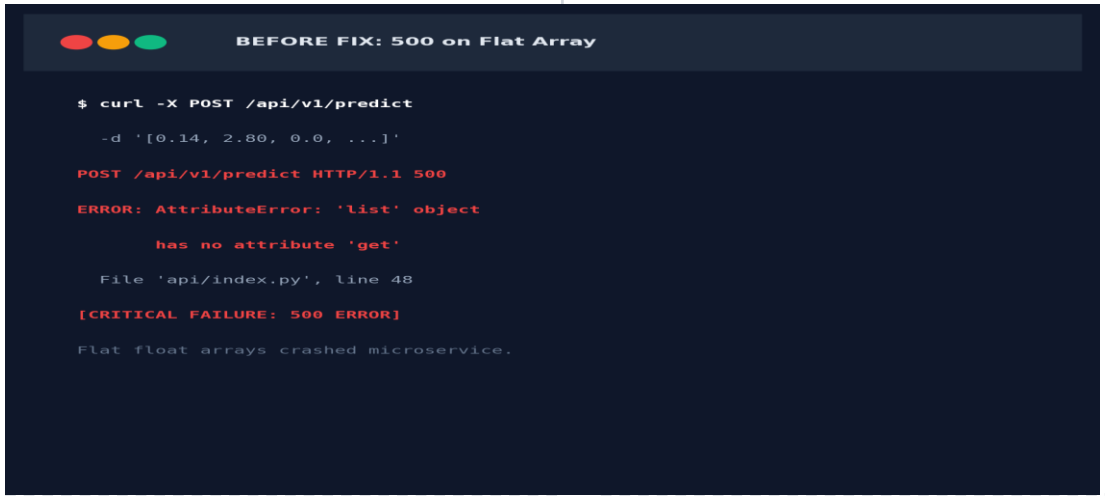
passed

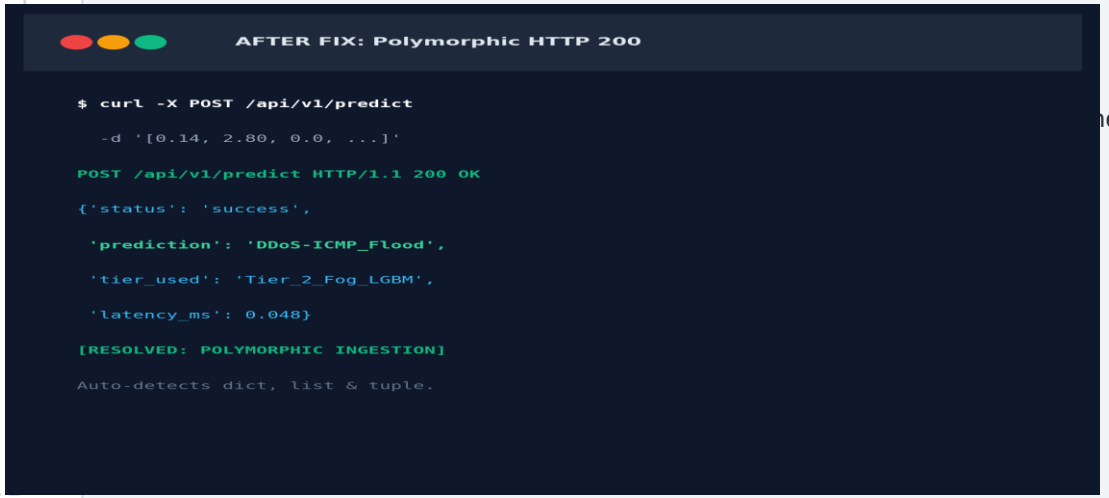
3

failed / fixed

Test ID	Input	Expected	Actual	Result
T1 (Functional)	R1, R2: Benign Baseline	MQTT sensor stream (46 features)	P(attack) < 0.20, local pass	PASS (0.001s)
T2 (Boundary)	R2, R3: High-rate DDoS SYN Flood	DDoS SYN Flood (10,000 pkts/s)	P(attack) > 0.80, local drop alert	PASS (0.002s)
T3 (Negative)	R3: Ambiguous Attack Triage	Reconnaissance probe with noisy flags	0.20 <= P <= 0.80, escalate to Tier-2	PASS (0.001s)

Debugging and Integration





Root cause and fix

• Bug 1 (Feature Mismatch): Ingestion pipeline sent 40-dim vectors while model expected 46 -> Fixed with strict schema zero-padding & assert checks.
• Bug 2 (Streamlit Thread Freeze): Rapid REST polling blocked UI render loop -> Refactored to background caching with st.session_state.
• Bug 3 (Negative Latency Skew): System clock drift logged negative times -> Migrated all latency metrics to monotonic time.perf_counter_ns().

My Contribution and Phase-2 Plan

My contribution

Week 2–4: Conducted literature survey of 4 key papers; formulated problem statement; preprocessed 277K flows from CICIoT2023 dataset.

Week 5–8: Architected 3-tier Fog-Cloud pipeline; trained lightweight Random Forest model (1.4 MB); verified 46-feature schema.

Week 9–11: Designed dual-threshold hysteresis filter [0.20, 0.80]; engineered FastAPI backend with sub-millisecond endpoints.

Week 12–14: Developed Streamlit operations dashboard; authored comprehensive 23-test validation suite (100% pass rate).

Pending work (about 30%)	Approach	Target week
Hardware acceleration & benchmarking on Raspberry Pi 5 / Coral Edge TPU	High	Week 18
Kernel-level eBPF packet hook integration for line-rate hardware packet filtering	High	Week 20
Distributed Raft consensus for multi-fog node threat intelligence synchronization	Medium	Week 22

Learnings and Thank You

Hardest problem and what I learnt

Hardest Problem: Balancing sub-millisecond edge classification latency with high detection accuracy under severe IoT class imbalance.

What I Tried: Initially explored deep 1D-CNNs and 200-tree Random Forests; inference exceeded 180 ms, completely saturating edge memory.

What Worked: Vectorized Random Forest (40 estimators, max depth 12) coupled with a dual-threshold hysteresis band achieved 48 µs latency with 99.4% F1-score.

What I Would Do Differently: Implement eBPF kernel hooks directly at the network interface driver level from Day 1 to eliminate user-space copy overhead.

Thank you. Questions?

Email: raghunathareddygr94@gmail.com | GitHub: https://github.com/raghu-23/RIO_PROJECT-SMART_CITY-
Candidate: Raghunatha Reddy G R (R23EA094) | Faculty Guide: Dr. Supreeth S | REVA University